

# Unarchitected Metal: Portable Neural Inference and Safety-Gated CLINCH Persona Integration in a UCI Chess Engine

Manus AI

*Unchessed AI Research*

*Independent Engineering Research*

*Sandboxed Research Environment*

**Abstract**—Human-facing chess engines must balance playing strength, natural move choice, engaging positions, and realistic errors without compromising tactical correctness. This paper presents Unarchitected Metal, a codename migration and runtime architecture for the Unchessed UCI engine, together with a dedicated CLINCH persona adapter. The architecture centralizes CPU capability detection, dispatches AVX2/FMA and AArch64 NEON kernels through a cached registry, and preserves scalar fallbacks and legacy model compatibility. The CLINCH adapter ranks search-approved candidates using centipawn-loss safety, shallow reply concentration, policy naturalness, and engagement signals. A comprehensive native benchmark compared scalar fallback against runtime-dispatched kernels across four positions, three hash sizes, and three repeats. On the native x86-64 host, dispatch reached 1,404,292 median NPS versus 742,908 scalar NPS, a 89.03 percent throughput difference. An x86-64-v3 build produced 1,357,775 versus 777,416 median NPS, a 74.65 percent difference. AArch64 Linux and Apple Silicon targets passed workspace compilation; native ARM execution was unavailable in the sandbox. The results establish software correctness and workload throughput, not Elo improvement. The paper records the compatibility contract, benchmark methodology, persona algorithm, threats to validity, and a statistically appropriate follow-up protocol.

**Index Terms**—chess engine, portable inference, AVX2, FMA, NEON, policy network, persona adaptation, CLINCH, tactical safety

## I. INTRODUCTION

Chess engines traditionally optimize a single objective: maximize the evaluation of the selected move. A human-facing engine has a broader objective. It may need to match an opponent’s skill, respond to a blunder, defend an inferior position, or create a challenging game while remaining tactically sound. This requires an architecture in which the search, neural evaluator, policy prior, and persona controller have separate responsibilities.

This paper presents two related contributions in the Unchessed UCI engine. The first is **Unarchitected Metal**, the canonical codename for the compact neural architecture and its portable runtime. The Metal migration includes a stable package identifier, canonical Rust and Python module paths, legacy aliases, and a centralized CPU capability registry. The runtime selects AVX2/FMA kernels on x86 hardware and NEON kernels on AArch64 while retaining scalar implementations as the reference path.

The second contribution is a dedicated **CLINCH adapter** for persona-conditioned move selection. CLINCH is intended for drawish or strategically balanced positions where the engine should create practical pressure without making arbitrary mistakes. It combines a strict search-loss budget with probe-derived reply concentration, policy naturalness, and a bounded engagement signal. Unlike MATCH, CLINCH does not deliberately inject blunders; natural blunders remain a separate, phase-aware MATCH behavior.

The paper makes three distinctions. First, a code migration is not a new learned model. Second, a kernel-throughput benchmark is not an Elo result. Third, human-like selection is not permitted to override a forced tactical result. These distinctions are essential for a safe evaluation of the system.

## II. CONTRIBUTIONS

The work makes the following contributions:

- 1) A compatibility-preserving **Unarchitected Metal** architecture with canonical module, tool, artifact, and documentation names.
- 2) A cached CPU capability registry supporting x86 AVX2, FMA, SSE4.1, AArch64 NEON, and Apple Silicon target identification.
- 3) A dedicated safety-gated CLINCH adapter that operationalizes natural moves, engaging positions, and natural blunders without conflating their mechanisms.
- 4) A reproducible scalar-versus-dispatch benchmark suite and current measurements across native x86-64 and x86-64-v3 builds, with AArch64 compile validation.
- 5) An IEEE-style engineering report that separates correctness, throughput, playing strength, and human-likeness claims.

## III. RELATED WORK

### A. Neural policy and human-like play

Leela Chess Zero treats the policy as a probability distribution over actions that guides search, while the value head estimates the resulting position [1]. Maia instead learns human move distributions rather than optimal engine moves [2]. Maia-2 extends the approach with skill-aware conditioning and coherence across skill levels [3]. These systems motivate

a separation in Unchessed: the policy expresses naturalness, while search remains authoritative for tactics.

### B. NNUE and low-latency inference

Stockfish’s NNUE documentation emphasizes sparse features, incremental updates, simple low-precision layers, and efficient integer-oriented inference [4]. The Unarchitected Metal runtime follows the same engineering principle of keeping inference compact and dispatching specialized kernels only when their feature requirements are satisfied. Dataset construction and leakage-resistant validation are also important because exact-position overlap can make checkpoint selection misleading [5].

### C. Portable SIMD dispatch

Portable dispatch has two independent requirements: the binary must execute safely on machines that lack optional instructions, and the optimized implementation must preserve the scalar result within a declared numerical tolerance. Metal therefore centralizes feature detection in a process-cached registry and retains a scalar implementation for every dispatched operation. The registry is deliberately narrower than a general compiler multiversioning framework; it covers only kernels whose target-feature contracts are explicit in the runtime.

## IV. UNARCHITECTURED METAL ARCHITECTURE

### A. Codename and compatibility contract

The previous architecture name, Unarchitected v1, was ambiguous in source paths and documentation. The canonical identity is now Unarchitected Metal. Active implementation paths use `unarchitected_metal` and `unarchitected_metal_runtime`; active Python tools, configuration files, artifacts, benchmarks, and research documents follow the same naming convention.

Newly exported tensor packages use the eight-byte UNMETAL1 identifier. Existing packages use UNARCHV1. The loader accepts both identifiers, while the writer emits UNMETAL1. Version number, header size, section table layout, alignment, CRC checks, quantization metadata, and tensor schema are unchanged. This makes the codename migration a format-compatible identity change rather than an unverified model conversion.

The old Rust modules remain re-export shims. The old UCI option names remain accepted in parallel with the canonical `UnarchitectedMetalHint`, `UnarchitectedMetalHintExit`, `UnarchitectedMetalFile`, and `UnarchitectedMetalMinTime` options. A migration document records these mappings and the rationale for retaining the aliases.

### B. Runtime structure

The runtime has five layers:

### C. Capability registry

The registry uses a process-level `OnceLock`. On x86, it records the results of `is_x86_feature_detected!` for

TABLE I  
UNARCHITECTURED METAL RUNTIME LAYERS.

| Layer             | Responsibility                                                             | Safety property                              |
|-------------------|----------------------------------------------------------------------------|----------------------------------------------|
| Package parser    | Validate UNMETAL1 or legacy UNARCHV1 sections, shapes, alignment, and CRCs | Malformed packages fail closed               |
| Feature registry  | Detect AVX2, FMA, SSE4.1, NEON, and Apple Silicon target identity once     | No repeated hot-loop detection               |
| Kernel dispatch   | Select scalar, AVX2, AVX2/FMA, AVX2/SSE4.1, or NEON implementation         | Target features match kernel declarations    |
| Neural runtime    | Execute compact policy/value inference and produce root hints              | Numerical parity remains testable            |
| Alpha-beta search | Remain authoritative for tactical correctness and final safety             | Humanization cannot force a tactical failure |

AVX2, FMA, and SSE4.1. On AArch64, NEON is treated as part of the supported baseline and Apple Silicon is identified by the `aarch64-apple-darwin` target. The environment variable `UNCHESSSED_DISABLE_SIMD=1` forces all optional paths off for reproducible scalar benchmarking.

The exact dispatch contract is:

```
has_avx2()      -> AVX2 accumulator and integer kernels
has_avx2_fma()  -> AVX2 + FMA floating-point dot products
has_avx2_sse41() -> AVX2 + SSE4.1 softmax path
has_neon()      -> AArch64 NEON accumulator and dot products
is_apple_silicon() -> target identity for reporting and deployment policy
```

Every optimized function retains a scalar equivalent. The dispatcher is called at the operation boundary, not from each vector lane or loop iteration. The ARM implementation includes tail handling and scalar-equivalent activation clamping so vector width does not change the output contract.

## V. CLINCH PERSONA INTEGRATION

### A. Persona responsibilities

The Unchessed persona state machine separates behavior into five regimes. FULL bypasses policy humanization and preserves raw engine strength. MATCH uses a policy prior and a phase-aware loss schedule. PUNISH prioritizes forcing moves after an opponent error. DEFEND maximizes resistance within a narrow safety budget. CLINCH selects a probe-qualified, natural, and difficult line in a strategically balanced position.

### B. CLINCH objective

For a candidate move  $m$ , let  $L(m)$  denote centipawn loss from the best search line,  $G(m)$  denote the shallow probe

TABLE II  
PERSONA RESPONSIBILITIES AND POLICY BOUNDARIES.

| Mode   | Primary objective                 | Policy role                                  |
|--------|-----------------------------------|----------------------------------------------|
| FULL   | Raw search strength               | Bypassed                                     |
| MATCH  | Level matching and natural errors | Strong naturalness prior                     |
| PUNISH | Convert opponent mistakes         | Tie-breaker among forcing moves              |
| DEFEND | Maximum resistance                | Small guide influence                        |
| CLINCH | Safe practical pressure           | Naturalness among probe-qualified candidates |

reply-concentration gap,  $P(m)$  denote the policy prior, and  $E(m)$  denote the CLINCH engagement bonus. The adapter ranks candidates using

$$S(m) = -L(m) + 0.60G(m) + 10 \ln(P(m) + \varepsilon) + E(m) \quad (1)$$

subject to

$$0 \leq L(m) \leq 40 \quad (2)$$

cp.

The adapter considers at most the top five search lines. It probes each surviving candidate after the move and falls back to the best search move if no line meets the 40-centipawn budget.

The CLINCH engagement signal is deliberately distinct from MATCH’s engagement score. If the shallow probe returns between 3 and 20 replies, CLINCH receives +8.0. If both queens survive the candidate move, it receives an additional +6.0. The signal is based on the **probe’s reply count**, not the resulting position’s raw legal-move count. MATCH uses a separate phase-aware engagement and blunder path.

### C. Three golden rules

The implementation treats humanization as three bounded mechanisms:

- 1) **Natural moves:** policy priors prefer moves present in human-like distributions, but only among search-approved candidates.
- 2) **Engaging positions:** reply concentration and queen-tension bonuses favor positions that leave a meaningful practical decision.
- 3) **Natural blunders:** MATCH may sample plausible loss-band candidates with opening, middlegame, and endgame multipliers; CLINCH never deliberately corrupts its safety boundary.

A natural blunder is therefore not a random legal move. It is a policy-supported candidate in a calibrated loss band, and forced mate or catastrophic tactical failures remain excluded.

## VI. BENCHMARK METHODOLOGY

### A. Workload and controls

The benchmark harness sends UCI commands to the release binary, waits for `bestmove`, and accepts only completed node-limited searches. All runs use one thread, fixed evaluator bytes, Adaptive disabled, opening book disabled, 500,000

TABLE III  
AGGREGATE SCALAR-VERSUS-DISPATCH THROUGHPUT.

| Build         | Scalar median NPS | Dispatch median NPS | Difference | Runs    |
|---------------|-------------------|---------------------|------------|---------|
| Native x86-64 | 742,908           | 1,404,292           | +89.03%    | 36 + 36 |
| x86-64-v3     | 777,416           | 1,357,775           | +74.65%    | 36 + 36 |

TABLE IV  
NATIVE x86-64 MEDIANS AT THE AGGREGATE HASH-SIZE COMPARISON.

| Position       | Scalar median NPS | Dispatch median NPS | Difference |
|----------------|-------------------|---------------------|------------|
| Start position | 782,627           | 1,466,451           | +87.38%    |
| Kiwipete       | 666,220           | 1,175,468           | +76.44%    |
| Middlegame     | 722,773           | 1,306,969           | +80.83%    |
| Endgame        | 892,960           | 1,485,827           | +66.39%    |

requested nodes, four positions, three hash sizes, and three repeats. The positions are start position, Kiwipete, a tactical middlegame, and a reduced-material endgame. The evaluator is a deterministic valid UNCHNNUE v1 fixture used solely for repeatable kernel throughput; it is not a production trained model.

The scalar condition sets `UNCHEDD_DISABLE_SIMD=1`. The dispatch condition uses the runtime registry. The x86-64-v3 condition additionally compiles with `-C target-cpu=x86-64-v3`. This makes the comparison a kernel-throughput measurement rather than a playing-strength test.

### B. Native x86-64 results

The native x86-64 host completed 36 scalar and 36 dispatched measurements. The aggregate median was 742,908 NPS for scalar fallback and 1,404,292 NPS for runtime dispatch.

The per-position native results are:

### C. ARM and Apple Silicon targets

The AArch64 Linux and Apple Silicon workspace checks passed after the Metal migration. The x86 sandbox has no QEMU AArch64 runner and no Apple Silicon hardware, so no native NEON NPS claim is made. The result is a compile and dispatch-contract validation, not a physical-hardware performance measurement.

## VII. SOFTWARE VALIDATION

The Rust core suite passed 132 tests with no failures and six existing ignored performance tests. The workspace release build passed. The migrated Metal Python suite passed 33 tests, with five skips caused by optional PyTorch calibration dependencies. A direct package compatibility check confirmed that a package emitted with `UNMETAL1` can be read after its header is changed to legacy `UNARCHV1`; the parser accepts both. The UCI option parser accepts canonical Metal names and legacy names.

TABLE V  
ARM TARGET VALIDATION STATUS.

| Target                    | Validation                   | Inter-<br>pre-<br>ta-<br>tion                   |
|---------------------------|------------------------------|-------------------------------------------------|
| aarch64-unknown-linux-gnu | Workspace cargo check passed | NEON code and scalar fallback compile           |
| aarch64-apple-darwin      | Workspace cargo check passed | Apple Silicon con-<br>target path com-<br>piles |
| Native Apple Silicon      | Not available                | Requires an M-series runner                     |
| Native ARM Linux          | Not available                | Requires AArch64 hardware or QEMU               |

TABLE VI  
SOFTWARE AND CROSS-TARGET VALIDATION.

| Validation item     | Result               | Meaning                              |
|---------------------|----------------------|--------------------------------------|
| Rust core tests     | 132 passed           | No functional regression detected    |
| Rust release build  | Passed               | Workspace binaries linked            |
| Metal Python tests  | 33 passed, 5 skipped | Skipped tests require optional Torch |
| UNMETAL1 emission   | Passed               | Canonical writer path works          |
| UNARCHV1 loading    | Passed               | Legacy checkpoints remain loadable   |
| AArch64 Linux check | Passed               | NEON target compiles                 |
| Apple Silicon check | Passed               | Apple target compiles                |

## VIII. THREATS TO VALIDITY

The benchmark uses a deterministic evaluator fixture rather than the full trained deployment artifact. This is appropriate for repeatable kernel comparison but does not establish model quality or playing strength. The native results were collected on one x86-64 machine; x86-64-v3 is a compiler-target comparison on the same host, not a survey of all v3 processors. ARM results are compile-only because the sandbox lacks AArch64 execution.

NPS is reported from completed UCI search information and is affected by search-tree shape, hash size, operating-system scheduling, and millisecond timing granularity. The scalar forcing environment is useful for isolating optional kernels, but it also changes the evaluator path by disabling all optional vector implementations. No performance number should be interpreted as a universal speedup for every workload.

The CLINCH coefficients are engineering anchors, not statistically optimized parameters. The six-game Stockfish Elo-capped smoke campaign documented in the earlier research record is not merged into these kernel results and is not an Elo estimate. A credible playing-strength study requires fixed binaries, production evaluator artifacts, paired colors, opening balancing, and a pre-registered SPRT.

## IX. REPRODUCIBILITY PROTOCOL

The committed benchmark scripts reproduce the scalar and dispatch conditions:

```
python3 scripts/make-benchmark-nnue.py
benchmarks/artifacts/benchmark-
v1.unchnnue
cargo build --workspace --release
python3 scripts/benchmark-dispatch.py \
--binary target/release/unchessed-
adapter \
--output benchmarks/results/
dispatch.csv \
--label native-dispatch --nodes 500000
--repeats 3 --hash 4 16 64
python3 scripts/benchmark-dispatch.py \
--binary target/release/unchessed-
adapter \
--output benchmarks/results/scalar.csv \
--label native-scalar --scalar --nodes
500000 --repeats 3 --hash 4 16 64
```

For x86-64-v3, build with `RUSTFLAGS='-C target-cpu=x86-64-v3'` and a separate target directory. For ARM, execute the same harness on native AArch64 hardware; a cross-target cargo check is not a replacement for a NEON runtime benchmark.

For CLINCH, the recommended evaluation compares no policy, MATCH-only policy, the dedicated CLINCH adapter, and indiscriminate policy use. Report tactical failures as hard rejects. Report naturalness, reply entropy, queen retention, probe reply count, centipawn loss, and phase-specific blunder rate separately. Do not promote a default based on NPS or a tiny game sample.

## X. CONCLUSION

Unarchitected Metal provides a coherent identity and portability boundary for the Unchessed neural architecture. It centralizes ISA detection, supports AVX2/FMA and AArch64 NEON code paths, preserves scalar reference behavior, and keeps legacy package and UCI interfaces loadable. On the available x86 host, the dispatched kernels substantially increased throughput over the forced scalar path under the controlled fixture workload. ARM and Apple Silicon support is compile-validated but still requires native hardware measurement.

The CLINCH adapter complements this runtime architecture by keeping persona behavior above the evaluator and below tactical authority. It selects natural, engaging, probe-qualified moves inside a 40-centipawn safety budget, while MATCH remains responsible for calibrated natural blunders. The combined design is ready for a stronger empirical campaign, but the correct next step is a production-artifact benchmark and paired-game study rather than an unconditional default change.

## REFERENCES

- [1] Leela Chess Zero Development Team, “Technical Explanation of Leela Chess Zero.” [Online]. Available: <https://lczero.org/dev/wiki/technical-explanation-of-leela-chess-zero/>
- [2] R. McIlroy-Young, S. Sen, J. Kleinberg, and A. Anderson, “Aligning Superhuman AI with Human Behavior: Chess as a Model System,” in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2020. [Online]. Available: <https://www.cs.toronto.edu/~ashton/pubs/maia-kdd2020.pdf>
- [3] Z. Tang, D. Jiao, R. McIlroy-Young, J. Kleinberg, S. Sen, and A. Anderson, “Maia-2: A Unified Model for Human-AI Alignment in Chess,” *arXiv preprint arXiv:2409.20553*, 2024, [Online]. Available: <https://arxiv.org/html/2409.20553v1>
- [4] Stockfish Development Team, “NNUE.” [Online]. Available: <https://official-stockfish.github.io/docs/nnue-pytorch-wiki/docs/nnue.html>
- [5] D. Tan and N. Watkinson Medina, “Study of the Proper NNUE Dataset,” *arXiv preprint arXiv:2412.17948*, 2024, [Online]. Available: <https://arxiv.org/html/2412.17948v1>